



Assignment No. 02

Computer Organization & Assembly Language (EE-2003)

ID: _____

NAME: _____

SECTION: _____

Read the Instructions Carefully

- ❖ Assignments are designed for your learning copying will lead to plagiarism. Practicing the questions yourself will help you in your exams. Fill the memory and register tables in the Word file first before running the respective programs in visual studio.
- ❖ The filled memory should represent the memory status of the executed program.
- ❖ Memory and register values should be in hexadecimal format.
- ❖ Each row in the memory table consists of 16 columns, each of 1 byte. The row indicates the starting offset. Both the starting offset and the columns are represented with hexadecimal values.
- ❖ Your assignment marks will be calculated from your submission (60%) + Quiz (30%) + Sessional-I (10%)
- ❖ You are supposed to debug each question in Visual Studio after filling the tables.
- ❖ You are supposed to submit a **SNAPSHOT** of your Visual Studio debugging screen, such that it shows the status of registers and memory after debugging. Your submission must include snapshots of each question with a visible time and date on your computer. The snapshots for each question must be attached in the Word file under the respective questions. Remember each snapshot has marks.
- ❖ Submit a PDF File with the naming format 23i-1234_Section_CS
- ❖ If you find any **ERROR** in the program highlight the error and write the justification for the error. You must understand ERROR carefully. After identifying the error, you will ignore it by commenting it out and running the program. Also, the register values corresponding to the erroneous statements must be left blank. For example:
; mov al, 256 ERROR: 256 is out of range for an 8-bit register like `AL`. It should be within 0-255.
- ❖ Data Declarations highlighted in yellow must be replaced as instructed below:

✓ 'id1' represents the first character of your ID, 'id2' represents the second character, 'id3' and 'id4' represent the third and the fourth characters of your ID, respectively.

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE
ASSIGNMENT#2 (FALL 2024)

1. Write equivalent C code for the assembly code given below. Update the final value of registers in **HEX**

ASM Program

```
.data
    temp db 0
.code

main Proc
    mov ax,0
    mov ecx,5
    Outer:
    mov temp,cl
    mov ecx,3
    Inner:
        add ax,1
        Loop Inner
    mov cx,0
    mov cl,temp
    Loop Outer

main ENDP
END main
```

[illegible]

2. Update Flags after executing following code.

NOTE: **AND** performs a Boolean(bitwise) **AND** operation between each pair of matching bits in two operands and place result in the destination. **AND** instruction always clear overflow and carry flags. It modifies Sign, Zero and Parity flags.

<pre> mov al,0AEH and al,246 </pre>	AX		
	CX		

3. Update memory after executing code given below

ASM Program																
<pre> .data ary db 26 dup(?) arysize = \$ - ary ary_copy db arysize dup(0) endmem db 1 .code main Proc mov esi, OFFSET ary mov edi, OFFSET ary_copy mov ax, 000FFh mov ecx, arysize and al, 061h mov bl, al and bl, 0CFh ;masking L1: mov [esi], al mov [edi], bl inc esi inc edi inc al inc bl Loop L1 main ENDP END main </pre>																
MEMORY																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00404000																
00404010																
00404020																

4. Update memory after executing code given below

ASM Program																	
<pre> .data ary db 26 dup(?) arysize = \$ - ary ary_copy db arysize dup(0) endmem db 1 .code main Proc mov esi, OFFSET ary mov edi, OFFSET ary_copy mov ax, 0041h mov ecx, arysize mov bl, al or bl, 00100000b L1: mov [esi], al mov [edi], bl inc esi inc edi inc al inc bl Loop L1 main ENDP END main </pre>																	
MEMORY																	
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
00404000																	
00404010																	
00404020																	
00404030																	

5. Update Flags after executing following code

NOTE: OR performs a Boolean(bitwise) **OR** operation between each pair of matching bits in two operands and place result in the destination. **OR** instruction always clear overflow and carry flags. It modifies Sign, Zero and Parity flags.

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE
ASSIGNMENT#2 (FALL 2024)

<pre>mov al, 11100011b OR al, 00000100b ;Setting 3rd bit</pre>	Flags	Sign	
		Zero	
		Carry	
		Overflow	
		Parity	
		Auxiliary	
Rough Work			

6. Update register after each line of code and update Flags after executing **XOR AL,AH**

<pre>mov ax,0A593H xor ax,-1 xor ax,0 xor ax,-1 xor ax,ax mov ax, 05A37H xor al,0 xor ah,1 xor al,ah ;line will check parity ;for 16 bit integer</pre>	<table><tr><td>AX</td><td></td><td></td></tr><tr><td>AX</td><td></td><td></td></tr><tr><td>AX</td><td></td><td></td></tr><tr><td>AX</td><td></td><td></td></tr><tr><td>AX</td><td></td><td></td></tr><tr><td colspan="3"></td></tr><tr><td>AX</td><td></td><td></td></tr><tr><td>AI</td><td></td><td></td></tr><tr><td colspan="3"></td></tr><tr><td>AH</td><td></td><td></td></tr><tr><td>AL</td><td></td><td></td></tr></table>	AX			AX			AX			AX			AX						AX			AI						AH			AL			<table><tr><td>Sign</td><td></td></tr><tr><td>Zero</td><td></td></tr><tr><td>Carry</td><td></td></tr><tr><td>Overflow</td><td></td></tr><tr><td>Parity</td><td></td></tr><tr><td>Auxiliary</td><td></td></tr></table>	Sign		Zero		Carry		Overflow		Parity		Auxiliary	
AX																																															
AX																																															
AX																																															
AX																																															
AX																																															
AX																																															
AI																																															
AH																																															
AL																																															
Sign																																															
Zero																																															
Carry																																															
Overflow																																															
Parity																																															
Auxiliary																																															
Rough Work																																															

7. Write a program that finds parity of number given below? **HINT:** Use XOR and LOOP to find parity

```
4 .data
5     parity DQ 0A1B2C3D4E5F67890H
```

CODE:

[illegible]

INSTRUCTIONS: (Question 8-11) Perform each of the following operations on word size 2's complement numbers and update the answer and value of flags after performing the arithmetic. Perform all the steps in the "calculation" box, only filling the answer will not get any credit

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE
ASSIGNMENT#2 (FALL 2024)

8. Update flags after arithmetic instruction? Also state which of the following jumps will taken or not taken

<pre>.code mov ax,0FFFFh add ax,0FC70h jc l1 L1: jz L2 L2: jo L3 L3: js L4 L4: jp L5 L5:</pre>		TAKEN	NOT TAKEN												
	Jc														
	Jz														
	Jo														
	Js														
	jp														
	<u>Calculations</u>														
<table><tr><td colspan="2">FLAGS</td></tr><tr><td>Sign</td><td></td></tr><tr><td>Zero</td><td></td></tr><tr><td>Carry</td><td></td></tr><tr><td>Overflow</td><td></td></tr><tr><td>Parity</td><td></td></tr></table>				FLAGS		Sign		Zero		Carry		Overflow		Parity	
FLAGS															
Sign															
Zero															
Carry															
Overflow															
Parity															

9. Update flags after arithmetic instruction? Also state which of the following jumps will taken or not taken

<pre>.code mov ax,07B1Ah sub ax,0CEEBh jc l1 L1: jz L2 L2: jo L3 L3: js L4 L4: jp L5 L5:</pre>	<table><tr><td></td><td>TAKEN</td><td>NOT TAKEN</td></tr><tr><td>Jc</td><td></td><td></td></tr><tr><td>Jz</td><td></td><td></td></tr><tr><td>Jo</td><td></td><td></td></tr><tr><td>Js</td><td></td><td></td></tr><tr><td>jp</td><td></td><td></td></tr></table>		TAKEN	NOT TAKEN	Jc			Jz			Jo			Js			jp		
	TAKEN	NOT TAKEN																	
Jc																			
Jz																			
Jo																			
Js																			
jp																			
<table><tr><td colspan="2">FLAGS</td></tr><tr><td>Sign</td><td></td></tr><tr><td>Zero</td><td></td></tr><tr><td>Carry</td><td></td></tr><tr><td>Overflow</td><td></td></tr><tr><td>Parity</td><td></td></tr></table>	FLAGS		Sign		Zero		Carry		Overflow		Parity		<u>Calculations</u>						
FLAGS																			
Sign																			
Zero																			
Carry																			
Overflow																			
Parity																			

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE
ASSIGNMENT#2 (FALL 2024)

10. Update flags after arithmetic instruction? Also state which of the following jumps will taken or not taken

<pre>.code mov cx,0FBCDh add cx,0E4AAh jnc l1 L1: jnz L2 L2: jo L3 L3: js L4 L4: jnp L5</pre>		TAKEN	NOT TAKEN
	Jc		
	Jz		
	Jo		
	Js		
	jp		

FLAGS	
Sign	
Zero	
Carry	
Overflow	
Parity	

Calculations

11. Update flags after arithmetic instruction? Also state which of the following jumps will taken or not taken

<pre>.code mov bx,0FABDh add bx,0684Ah jc l1 L1: jz L2 L2: jno L3 L3: jns L4 L4: jp L5 L5: mov ah,04ch</pre>		TAKEN	NOT TAKEN															
	Jc																	
	Jz																	
	Jo																	
	Js																	
	jp																	
<table><tr><th colspan="2">FLAGS</th></tr><tr><td>Sign</td><td></td></tr><tr><td>Zero</td><td></td></tr><tr><td>Carry</td><td></td></tr><tr><td>Overflow</td><td></td></tr><tr><td>Parity</td><td></td></tr></table>				FLAGS		Sign		Zero		Carry		Overflow		Parity		<u>Calculations</u>		
FLAGS																		
Sign																		
Zero																		
Carry																		
Overflow																		
Parity																		

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE
ASSIGNMENT#2 (FALL 2024)

12. Update flags after CMP instruction? Also state which of the following jumps will taken or not taken

<pre>.code mov bx,0A5A5h not bx cmp bx,05A5Ah jz L1 L1: je L2 jnz L3 jne L4 L4:</pre>	<table border="1" style="width: 100%; border-collapse: collapse; text-align: center;"> <tr> <th style="width: 10%;"></th> <th style="width: 40%;">TAKEN</th> <th style="width: 50%;">NOT TAKEN</th> </tr> <tr> <td>Jc</td> <td></td> <td></td> </tr> <tr> <td>Jz</td> <td></td> <td></td> </tr> <tr> <td>Jo</td> <td></td> <td></td> </tr> <tr> <td>Js</td> <td></td> <td></td> </tr> <tr> <td>jp</td> <td></td> <td></td> </tr> </table>		TAKEN	NOT TAKEN	Jc			Jz			Jo			Js			jp		
	TAKEN	NOT TAKEN																	
Jc																			
Jz																			
Jo																			
Js																			
jp																			
<table border="1" style="width: 100%; border-collapse: collapse; text-align: center;"> <tr> <th colspan="2">FLAGS</th></tr> <tr> <td style="width: 15%;">Sign</td><td></td></tr> <tr> <td>Zero</td><td></td></tr> <tr> <td>Carry</td><td></td></tr> <tr> <td>Overflow</td><td></td></tr> <tr> <td>Parity</td><td></td></tr> </table>	FLAGS		Sign		Zero		Carry		Overflow		Parity		<u>Calculations</u>						
FLAGS																			
Sign																			
Zero																			
Carry																			
Overflow																			
Parity																			

RENAME JUMPS: JE= JZ , JNZ= JNE

13. Update value of **ax** and **cx** registers after every iteration. Update any changes to the done to flag

<pre>.code mov ecx,5 mov ax,1 L1: inc ax dec ecx jcxz end_loop jmp L1 end_loop:</pre>	<table border="1" style="width: 100%; border-collapse: collapse; text-align: center;"> <tr> <th style="width: 10%;"></th> <th style="width: 40%;">TAKEN</th> <th style="width: 50%;">NOT TAKEN</th> </tr> <tr> <td>Jc</td> <td></td> <td></td> </tr> <tr> <td>Jz</td> <td></td> <td></td> </tr> <tr> <td>Jo</td> <td></td> <td></td> </tr> <tr> <td>Js</td> <td></td> <td></td> </tr> <tr> <td>jp</td> <td></td> <td></td> </tr> </table>		TAKEN	NOT TAKEN	Jc			Jz			Jo			Js			jp																	
	TAKEN	NOT TAKEN																																
Jc																																		
Jz																																		
Jo																																		
Js																																		
jp																																		
<table border="1" style="width: 100%; border-collapse: collapse; text-align: center;"> <tr> <th colspan="2">FLAGS</th></tr> <tr> <td style="width: 15%;">Sign</td><td></td></tr> <tr> <td>Zero</td><td></td></tr> <tr> <td>Carry</td><td></td></tr> <tr> <td>Overflow</td><td></td></tr> <tr> <td>Parity</td><td></td></tr> </table>	FLAGS		Sign		Zero		Carry		Overflow		Parity		<u>Calculations</u> <table border="1" style="width: 100%; border-collapse: collapse; text-align: center; margin-top: 20px;"> <tr> <th style="width: 10%;"></th> <th style="width: 12.5%;">1</th> <th style="width: 12.5%;">2</th> <th style="width: 12.5%;">3</th> <th style="width: 12.5%;">4</th> <th style="width: 12.5%;">5</th> <th style="width: 12.5%;">6</th> </tr> <tr> <td>CX</td> <td></td><td></td><td></td><td></td><td></td><td></td> </tr> <tr> <td>AX</td> <td></td><td></td><td></td><td></td><td></td><td></td> </tr> </table>		1	2	3	4	5	6	CX							AX						
FLAGS																																		
Sign																																		
Zero																																		
Carry																																		
Overflow																																		
Parity																																		
	1	2	3	4	5	6																												
CX																																		
AX																																		

INSTRUCTIONS: (Question 14) *CMP performs implied subtractions of a source operand from destination operand and updates flags, after CMP neither operands are modified.*

- For unsigned operation you will require **Zero** and **Carry Flag**. Flags will determine relationship between destination and source given in table below
 - **Note:** To calculate flags after **SUB** command use binary subtraction where **Carry Flag** act as borrow flag whereas **AUX** flag act as **Aux Borrow** for least nibble

CMP Results	ZF	CF
Destination < source	0	1
Destination > source	0	0
Destination = source	1	0

- For signed operation you will require **Zero**, **Overflow** and **Sign Flag**. Flags will determine relationship between destination and source given in table below
- **Note:** To calculate flags after **SUB** command, use 2's complement subtraction where overflow is calculated using **XOR** operation

CMP Results	Flags
Destination < source	SF $\neq$ OF
Destination > source	SF = OF
Destination = source	ZF = 1

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE
ASSIGNMENT#2 (FALL 2024)

14. Update flags after every **CMP** instruction and calculate jumps

MOV al,+127 CMP al,-128 JA IsAbove JG IsGreater		TAKEN	NOT TAKEN
	JA		
	JG		
<u>Subtraction to Calculate Flags for unsigned numbers</u>		<u>2's Complement addition to calculate flags for signed number</u>	

MOV ax,-1 CMP ax,0 JNL L1 JNLE L2 JL L4 JB L4		TAKEN	NOT TAKEN
	JNL		
	JNLE		
	JL		
	JB		
<u>Subtraction to Calculate Flags for unsigned numbers</u>		<u>2's Complement addition to calculate flags for signed number</u>	

MOV ax,+32 CMP ax,-35 JNG L1 JNGE L2 JGE L3 JNAE L4		TAKEN	NOT TAKEN
	JNG		
	JNGE		
	JL		
	JB		
<u>Subtraction to Calculate Flags for unsigned numbers</u>		<u>2's Complement addition to calculate flags for signed number</u>	

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE
ASSIGNMENT#2 (FALL 2024)

MOV cx,0 CMP cx,0 JG L1 JNG L2 JB L3 JNB L4 JLE L5		TAKEN	NOT TAKEN
	JNG		
	JNG		
	JB		
	JNB		
	JLE		
<u>Subtraction to Calculate Flags for unsigned numbers</u>		<u>2's Complement addition to calculate flags for signed number</u>	

MOV ax,-12 CMP cx,12 JB L1 JNAE L2 JNGE L3 JL L4		TAKEN	NOT TAKEN
	JB		
	JNAE		
	JNGE		
	JL		
<u>Subtraction to Calculate Flags for unsigned numbers</u>		<u>2's Complement addition to calculate flags for signed number</u>	

MOV ax,12 CMP cx,-12 JA L1 JNBE L2 JNLE L3 JG L4		TAKEN	NOT TAKEN
	JA		
	JNBE		
	JNLE		
	JG		
<u>Subtraction to Calculate Flags for unsigned numbers</u>		<u>2's Complement addition to calculate flags for signed number</u>	

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE
ASSIGNMENT#2 (FALL 2024)

Example ID		23I - 3812
First digit of Your ID	ID1	3
Second digit of Your ID	ID2	8
Third digit of Your ID	ID3	1
Fourth digit of Your ID	ID4	2

INSTRUCTIONS: (Question 15-16) Dry run the program for each iteration with following consideration

15. Dry run program gives below update table for each iteration count total number of iterations took by the program

ASM Program	
<pre>.data data dw 66h, id1, 045h, id2, 040h, id3, -025h, id4, -010h, 011h swap db 0 .code main PROC Start: mov swap,0 ;Reset swap flag to no swap mov ebx,0 ;initialize array index to zero loop1: mov ax,[ebx+data] ;load number to ax cmp ax,[ebx+data+2] ;compare with next number jbe noswap ;no swap if already inorder mov dx,[ebx+data+2] ;load second element in dx mov [ebx+data+2],ax ;store first number in second mov [ebx+data],dx ;store second number in first mov swap,1 ;flag that a swap has been done noswap: add bx,2 ;advance bx to next index cmp bx,18 ;are we at last index jne loop1 ;if not than compare next 2 cmp swap,1 ;check if swap has been done je start ;if yes than make another pass INVOKE ExitProcess,0 main ENDP END main</pre>	

ASSIGNMENT#2 (FALL 2024)

[illegible]

ASSIGNMENT#2 (FALL 2024)

[illegible]

16. Dry run program gives below update table for each iteration count total number of iterations took by the program

ASM Program	
.data	
data dw 66h, id1, 045h, id2, 040h, id3, -025h, id4, -010h, 011h	
swap db 0	
.code	
main PROC	
start:	
mov swap,0	;Reset swap flag to no swap
mov ebx,0	;initialize array index to zero
loop1:	
mov ax,[ebx+data]	;load number to ax
cmp ax,[ebx+data+2]	;compare with next number
jge noswap	;no swap if already inorder
mov dx,[ebx+data+2]	;load second element in dx
mov [ebx+data+2],ax	;store first number in second
mov [ebx+data],dx	;store second number in first
mov swap,1	;flag that a swap has been done
noswap:	
add bx,2	;advance bx to next index
cmp bx,18	;are we at last index
jne loop1	;if not than compare next 2
cmp swap,1	;check if swap has been done
je start	;if yes than make another pass
INVOKE ExitProcess,0	
main ENDP	
END main	

ASSIGNMENT#2 (FALL 2024)

[illegible]

ASSIGNMENT#2 (FALL 2024)

[illegible]

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE
ASSIGNMENT#2 (FALL 2024)

17. Modify the program in **QUESTION#16** for the following data declaration. Consider array as signed

```
data dd 66h, id1, 045h, id2, 040h, id3, -025h, id4, -010h, 011h
swap db 0
```

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE
ASSIGNMENT#2 (FALL 2024)

18. Fill and clear stack using following code. Fill given memory at the end of code as well. Stack starts at **0x000000100**.

.data
string1 BYTE "Computer Organization & Assembly language",0
len_str1 = (\$ - string1 - 1)

.code	.stack	
main PROC		
mov ecx, len_str1	00EA	00C2
mov esi, OFFSET string 1		
L1:	00EC	00C4
mov eax, 0		
mov al, [esi]	00FE	00C6
inc esi		
PUSH eax	00F0	00C8
LOOP L1		
	00F2	00CA
mov ecx, len_str1		
mov esi, OFFSET string 1	00F4	00CC
L2:		
mov eax, 0	00F6	00CE
pop eax		
mov [esi], byte PTR al	00F8	00E0
inc esi		
LOOP L2	00FA	00E2
	00FC	00E4
	00FE	00E6
	0100	00E8

MEMORY																	
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
00404000																	
00404010																	
00404020																	
00404030																	

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE
ASSIGNMENT#2 (FALL 2024)

19. Dry run the assembly code given below and update flag register after executing POPF

<pre> .data Wvall WORD 0FFFFH saveFlag WORD 0 .code mov ax, 0 mov ax, Wvall add ax,1 PUSHF ;store flag register POP saveFlag ;Cannot move flag register ;value to other register/or to memory ;Push Flag & pop to local could be used mov bx,saveFlag PUSHF mov cx, 0FFFFH add cx, 1 ;will update flag register POPF </pre>	<table border="1"> <tr><td>Sign</td><td></td></tr> <tr><td>Zero</td><td></td></tr> <tr><td>Carry</td><td></td></tr> <tr><td>Overflow</td><td></td></tr> <tr><td>Parity</td><td></td></tr> <tr><td>Calculation</td><td></td></tr> </table>	Sign		Zero		Carry		Overflow		Parity		Calculation	
Sign													
Zero													
Carry													
Overflow													
Parity													
Calculation													

20. Update stack after running assembly code given below. Fill stack memory after PUSH/POPA

.code	.stack		
mov ax,1	00F0		
mov cx,2			
mov dx,3	00F2		
mov bx,4			
mov bp,5	00F4		
mov si,6			
mov di,7	00F6		
PUSHA			
mov ax,15	00F8		
mov cx,14			
mov dx,13	00FA		
mov bx,12			
mov bp,11	00FC		
mov si,10			
mov di,9	00FE		
POPA			

MEMORY																	
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
00404000																	
00404010																	
00404020																	

21. Update stack after running assembly code given below and how return addresses are saved

22. Update stack after running assembly code given below and fill stack memory given below

[illegible]

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE
ASSIGNMENT#2 (FALL 2024)

23. Update stack after running assembly code given below

```
10 fact proc
11     PUSH Bp
12     mov bp,sp
13     mov ax,[bp+4]
14     cmp ax,0
15     ja l1
16     mov ax,1
17     jmp l2
18     mul bx
19     L1:
20     dec ax
21     push ax
22     call fact
23     return:
24     mov bx,[bp+4]
25     mul bx
26     l2:
27     pop bp
28     RET 2
29 fact ENDP
```

```
30 main proc
31     mov ax,0
32     PUSH CX
33     PUSH DX
34     PUSH AX
35     mov ax,A0
36     push ax
37     call fact
38 main EndP
```

FILL STACK		STACK AT END OF PROGRAM	
00C6		00C6	
00C8		00C8	
00CA		00CA	
00CC		00CC	
00CE		00CE	
00E0		00E0	
00E2		00E2	
00E4		00E4	
00E6		00E6	
00E8		00E8	
00EA		00EA	
00EC		00EC	
00EE		00EE	
00F0		00F0	
00F2		00F2	
00F4		00F4	
00F6		00F6	
00F8		00F8	
00FA		00FA	
00FC		00FC	
00FE		00FE	

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.